The procedural language is available as an extension. By running from psql `\dx` you'll see the `plpgsql` extension listed.

For SQL Functions, PostgreSQL calls these Query Language (SQL) functions.[4] SQL Functions have some capabilities that are similar to writing Ruby methods like variable numbers of arguments and default arguments. It's worth starting out with SQL functions because they're less complicated, and then moving to PL/pgSQL if you need to set variables and have conditional branching and loops in your functions.

Start with a skeleton structure without an implementation so that you can learn the basics.

The *signature* of the function accepts a single argument. The argument must be a `varchar` type and return a `varchar` text string.

Run psql from your terminal and type out this function structure:

sql/scrub_email_v1.sql

```
CREATE OR REPLACE FUNCTION scrub_email(email_address varchar(255))
RETURNS varchar(255) AS $$
SELECT
  email_address;
$$ LANGUAGE SQL;
```

`CREATE OR REPLACE FUNCTION` creates a named function or overwrites one that was defined with the same name and arguments. The function accepts an argument `email_address` with the type `varchar(255)`. The argument type is important because your argument will be database column values with specific types, and the column type must match the accepted argument type exactly.

This function doesn't do anything yet. Notice it just calls `SELECT` on your input value.

Once you've pasted your function body into psql, you can now call your function using `SELECT`:

sql/select_scrub_email.sql

```
SELECT scrub_email(email) FROM users;
```

Nice! That's the basics of a SQL function. You can use functions to build up SQL statements. You can even take things to another level with variables and loops.

Functions you define are source code just like your Rails application code, so you'll want to keep them in version control and manage their lifecycle.

4. https://www.postgresql.org/docs/current/xfunc-sql.html